School of Engineering

Institute of Mechatronic Systems (IMS)

Project thesis System Engineering

Betaflight – Offline Controller Tuning

Author(s)		Yuri Bianchi Janick Dort Dario Jurietti
Main supervisor		Michael Ernst Peter
Secondary supervisor		Ruprecht Altenburger
Date		19.12.2025

1 Summary

Dieses Projekt befasst sich mit der Entwicklung eines Werkzeugs zur Offline-Abstimmung von Gyroskop-Reglern für FPV-Racing-Drohnen auf Basis der Open-Source-Flugsteuerungssoftware Betaflight. Die Qualität des Gyro-Regelkreises beeinflusst direkt die Flugstabilität, die Unterdrückung von Störungen wie Propwash sowie die Wendigkeit der Drohne. Bisher erfolgt die Abstimmung der Drehraten-Regler meist durch zeitaufwendiges Ausprobieren.

Ziel des Projekts ist die Entwicklung einer modularen Open-Source-Lösung zur Offline-Optimierung auf Grundlage eines bestehenden Funktionsmusters. Die Umsetzung erfolgt in MATLAB und wird durch eine schlanke Python-Version ergänzt. Zusätzlich werden eine ausführliche Dokumentation im Markdown-Format sowie Beispielanwendungen auf GitHub bereitgestellt. Das Projekt soll der Gemeinschaft eine einfache Möglichkeit bieten, Drohnenparameter effizient zu optimieren und dadurch die Flugleistung zu verbessern.

Die Ergebnisse werden in einem öffentlich zugänglichen GitHub-Repository veröffentlicht und in einem Bericht dokumentiert. Neben der Softwareentwicklung beinhaltet das Projekt auch praktische Versuche mit FPV-Drohnen zur Überprüfung der entwickelten Methoden.

2 Abstract

This project focuses on the development of a tool for offline tuning of gyro controllers for FPV racing drones based on the open-source firmware Betaflight. The quality of the gyro control loop has a direct influence on flight stability, the suppression of disturbances such as prop wash, and the agility of the drone. Until now, rate controller tuning has mostly been performed through time-consuming trial and error.

The aim of the project is to develop a modular, open-source solution for offline tuning based on an existing proof of concept. The code will be implemented in MATLAB, accompanied by a lightweight Python version, and supplemented with detailed Markdown documentation and examples on GitHub. The project aims to provide the community with a simple way to efficiently optimise drone parameters and thereby improve flight performance.

The results will be published in a public GitHub repository and documented in a report. In addition to the software development, the project also includes practical work with FPV drones to validate the developed methods.

3 Preface

This document summarizes the results of our project thesis, which we worked on during the fall semester 2025 at the ZHAW School of Engineering. For us as systems engineering students, this project provided an ideal opportunity to apply advanced control theory and signal processing methods to a real, highly dynamic system. FPV drones present an interesting engineering challenge due to their highly dynamic behaviour and the need for precise control. This motivated us to investigate mathematical modelling, control engineering aspects, and offline tuning methods aimed at improving flight performance.

Our motivation was to translate the theoretical control concepts we learned over the past three years to a real, highly dynamic system. We also aimed to develop a tool that not only meets engineering standards but also provides real value to the FPV community by replacing subjective tuning with objective data analysis.

This work was carried out at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. We would like to express our gratitude to our supervisor, Michael Peter, for his guidance and support throughout this project. His expertise, constructive feedback, and willingness to take the time to assist us in developing complex algorithms were invaluable.

Contents

4 Introduction

First-person-view (FPV) racing drones require precise and responsive control to achieve high flight stability, minimize disturbances such as prop wash, and maintain agility during rapid manoeuvres. Proper tuning of the rate controllers is key to this performance, as it directly determines how the drone reacts to pilot inputs and external influences. Until now, tuning these controllers has mainly relied on iterative trial-and-error methods, a process that is both time-consuming and difficult to perform, especially for less experienced pilots.

Betaflight, an open-source firmware widely used in the FPV community, provides a flexible platform for configuring flight controllers. However, despite its popularity, efficient tuning remains a challenge. To address this, our project builds upon an existing proof of concept for offline tuning of rate controllers using flight data and system identification techniques. The goal is to transform this concept into a robust, modular, and open-source library that simplifies the tuning process.

The library will be implemented in 'MATLAB and Python', offering tools for analyzing flight logs, estimating system dynamics, and optimizing controller parameters without requiring repeated flight tests. A documentation is created, which shows how to use the tool and will be published on GitHub to ensure accessibility for the FPV community.

Besides software development, the project involves also building an FPV drone and testing and confirm the effectiveness of the proposed methods. By providing a structured, data-driven approach to tuning, this work aims to improve flight performance and reduce the barriers to achieving optimal control settings for both hobbyists and professionals.

4.1 Initial situation

The open-source firmware Betaflight is the most widely used platform for FPV racing drones and is continuously developed by a large community. The quality of the rate control is crucial for flight performance, as it enables stability, agility, and effective suppression of disturbances such as prop wash.

Today, tuning of controller parameters is still largely performed by trial-and-error adjustments, a method that is time-consuming and often challenging for less experienced pilots. A proof of concept (developed by Michael Peter) already exists, which enables offline tuning of rate controllers using flight measurement data obtained from chirp signal excitation. However, the current implementation is based on MATLAB, which is not widely used within the FPV community and therefore limits accessibility. In addition, the proof-of-concept requires a certain level of technical knowledge to operate, which further complicates its practical use for many pilots.

As a result, the potential of data-driven controller tuning remains largely unused in everyday FPV practice. This creates a gap between academically established system identification and control design methods on the one hand, and the predominantly experience-based tuning approaches used by pilots on the other. Bridging this gap represents a key motivation for the further development of the existing proof of concept.

4.2 Task and Objective

The aim of this project is to further develop the existing proof of concept into an accessible and practical tool for tuning rate controllers on Betaflight-based FPV drones. The focus lies on transforming the prototype from a research-oriented MATLAB implementation into a solution that

can be used by the wider community with minimal technical barriers. This includes designing a clean and modular software architecture, improving usability, and integrating functions for importing, processing, and analysing flight data.

In addition, the tool will be accompanied by comprehensive Markdown documentation, including explanations of key methods, usage instructions, and practical examples to support new users. The implementation will serve as a foundation for future developments, such as extended analysis features, alternative controller designs, or deployment in other embedded flight control systems. The overarching objective is to provide a reliable, data-driven workflow that replaces subjective trial-and-error tuning and enables verifiable control performance improvements.

5 Preparation

5.1 Chirp

5.1.1 Chirp Concept

A chirp is an input signal that sweeps continuously over a defined range (e.g. from a low frequency to a high frequency). This signal excites the system on multiple frequencies in one run and thus lets you observe the frequency response and step response, the foundation of controller tuning. Mathematically, a chirp signal can be represented as a sinusoidal wave with instantaneous frequency that varies linearly or exponential with time. For more information about the mathematical background, please have a look in the document Chirp Signal [1].

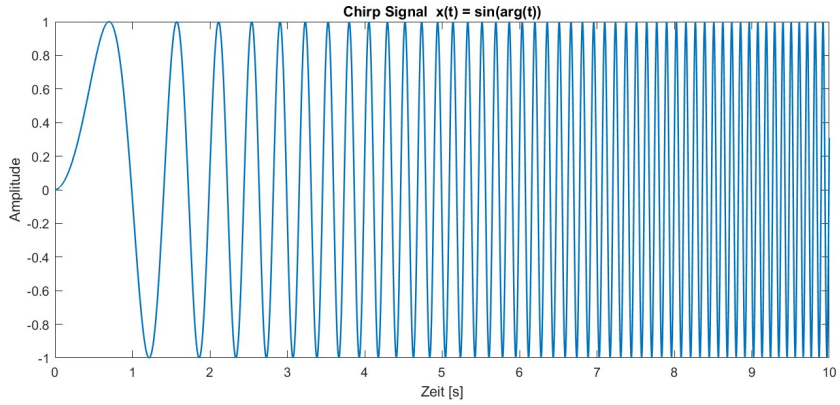

Figure 1: Chirp Signal example

5.1.2 Chirp on a drone

In the context of tuning a drone's flight controller, an exponential sweep is used. This is due to the dominant low-frequency dynamics, where low frequencies exhibit more predictable responses, allowing the sweep to allocate more time to higher frequencies for better resolution of system behavior. Additionally, multirotor vibration resonances are typically spaced logarithmically, making exponential sweeps more suited for efficient excitation across the frequency range.

The process involves these steps:

1. Signal Injection: The chirp signal is added to the pilot's stick input. This composite signal becomes the setpoint for the PID controller.
2. System Response: The drone attempts to follow this rapidly changing setpoint. The motors react, and the drone will oscillate at varying frequencies corresponding to the chirp signal.
3. Data Logging: While the chirp is active, high-resolution data from the gyroscope and the motor outputs are recorded using Betaflight's Blackbox feature. This data captures how the drone responds to the control inputs at different frequencies.

The recorded log file, containing the input chirp and the corresponding gyroscope output, is then used for offline analysis to determine the frequency response of the system.

5.1.3 Chirp programming

The implementation of the chirp signal in Betaflight is straightforward. You call the function chirpInit() and pass:

- a struct of type chirp_t defined in chirp.h
- starting frequency
- end frequency
- signal length
- loop-time in microseconds

This initialises the values in the data struct and:

- converts loop period from microseconds to seconds
- calculates the number of samples in the signal duration
- calculates the exponential growth factor
- precomputes the constants used to get the phase from the frequency evolution
- calls chirpReset() function

The chirpReset() function sets:

- internal counter to 0
- bool isFinished to false

and then calls the function chirpResetSignals() to set signal-related fields to 0.
Once the chirp is initialised use the chirpUpdate() function once per loop iteration.
This function checks:

- if bool isFinished is true -> return false
- else if internal counter is equal to the number of samples inside the chirps period -> set bool isFinished true, call chirpResetSignals(), return false

if none of the above is true the ...

6 Theory

6.1 Signal Processing

Signal processing converts raw Betaflight Blackbox flight data into useful information for analysis. These logs contain data such as stick movements, gyro signals and PID responses. Although the data may seem complex at first, tools like our analysis software and the Betaflight Blackbox Explorer help to make it easier to understand through clear visualisations for troubleshooting and tuning.

Preparing the data is a key step in the workflow. Before analysis, the signals are cleaned and brought into a consistent form, usually by filtering to reduce noise without changing the real flight behaviour. Based on the processed signals, the frequency response of the control system can be estimated to describe how it reacts to different inputs. This improves data quality and leads to more reliable results.

6.1.1 Data Import

The first step of any analysis is to load the measurement data correctly. A Betaflight Blackbox log has two main parts: the header and the data block. The header contains important information about the recording conditions, such as loop rates, logging settings, filter setup and PID values. Without this information, correct calculations are not possible and the data loses its value for analysis.

Importing the data is very computationally demanding. To save time, the logs are converted from raw CSV files into the faster .mat format. This avoids reprocessing the large files every time. The signal names (for example gyro and motor outputs) are also assigned automatically to the correct data columns. More details can be found in the Dataimport Documentation [2].

6.1.2 Handle High-Resolution Data

Betaflight offers an optional High Resolution Mode for logging, where selected signals are multiplied by ten before being saved. Since the data is stored as integers, this reduces the need for decimal values and limits rounding errors. During analysis, these signals must be scaled back to their original units; otherwise, frequency plots, tuning results and general interpretation will be incorrect. Key signals such as gyro data and control inputs are affected by this process. More details can be found in the Unscale high Resolution Gain documentation [3].

6.1.3 Time Conversion and Sampling Rate Verification

In a Betaflight log, time is stored as a rising counter in microseconds, not as regular seconds. Due to hardware limits or system load, the spacing between time stamps is not always constant. This step converts the time data into a uniform time base so that all samples can be compared correctly in time.

By checking the time differences between samples, logging issues such as dropped frames or small changes in the sampling rate can be detected. Correct time handling is essential for all later analysis steps, such as frequency calculations or controller tuning. More details are given in the Convert and evolution Time documentation. For drone tuning, see also the Evolution Time Flight description [3].

6.1.4 Application of Rotational Filtering (Apply Rotfiltfilt)

Filtering is a fundamental step in preparing the data for analysis, as it removes unwanted noise while preserving the original characteristics of the signals. The function `Apply Rotfiltfilt` operation applies a zero-phase filter, ensuring that the data's phase relationships remain intact while suppressing selected frequency components such as low-frequency drifts or high-frequency noise.

This step applies a forward-backward filtering technique, which processes the data in both forward and reverse directions and effectively cancels out phase distortions. This method is particularly useful when working with chirp signals, as it enables precise filtering without introducing phase shifts that could alter the signal's characteristics.

Proper application of this technique ensures that the data is clean and reliable for subsequent analysis steps. For more insights on how this method works, refer to the full description in the Apply Rotfiltfilt documentation [4].

6.1.5 Estimation of Frequency Response

The `Estimate Frequency Response` function uses advanced techniques to measure and analyze the relationship between a system's input and output signals in the frequency domain. This makes it possible to determine how different frequencies are amplified or attenuated by the system, providing valuable information about its dynamic behavior.

This technique is based on the Welch Method, which divides the signal into overlapping segments. By applying a Hann window to each segment and performing spectral averaging, the influence of noise is reduced and a smoother estimate of the frequency response is obtained. The result is a single-sided, amplitude-calibrated response curve that shows how individual frequencies are amplified or attenuated, as well as how phase shifts occur across the spectrum.

The resulting data is not only accurate but also robust, making it suitable for applications like control system tuning or stability analysis. For a deeper dive into the theoretical principles and implementation steps, consult the detailed Estimate Frequency Response documentation [5].

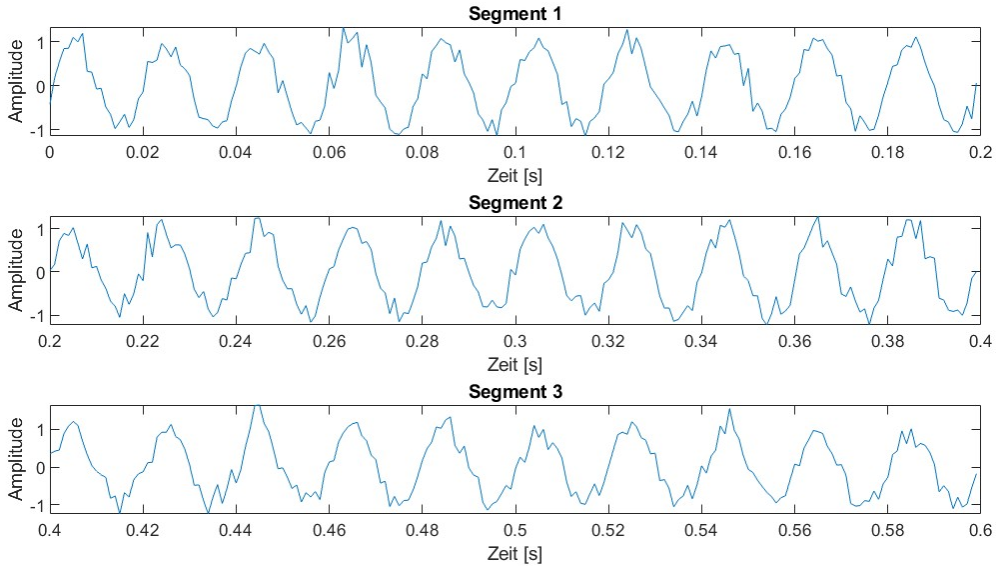

Figure 2: Segmentations of Data

6.2 Control System

The control system determines the dynamic response of a drone and is therefore central to flight stability and precise maneuvering. In this section, we analyse the individual components of the controller, examine their influence on the system behaviour and outline the methods used for tuning in order to improve performance and responsiveness.

6.2.1 Transfer Function Estimation

To analyse how the system reacts to control inputs, the transfer function of the plant is estimated directly from closed-loop flight data. This makes it possible to characterise the system dynamics without opening the control loop and to determine how input signals are transformed into outputs. The resulting model forms the basis for matching the controller parameters to the real system. More details can be found in the Estimate Transfer Function documentation.

6.2.2 Closed Loop Calculation

The interaction between the controller and the estimated transfer function of the plant determines the stability and robustness of the closed-loop system. By calculating the closed-loop transfer functions for both inner and outer loops, sensitivity, disturbance rejection, and overall system stability can be evaluated. These calculations provide a clear view of the controller dynamics and their impact on the system response. For details on the implementation, see the Calculate Closed Loop documentation. [6]

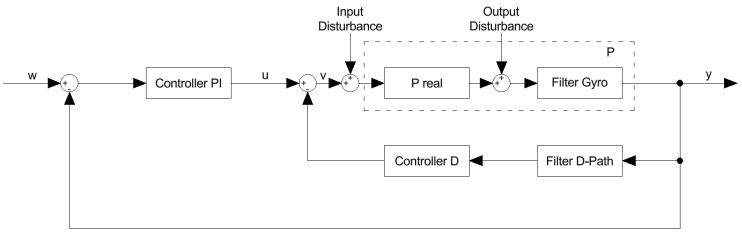

Figure 3: Gyro Control Loop

6.2.3 Filters and Controllers

Filters and controllers form the foundation of the control loop. Filters remove unwanted noise from sensor data or motor outputs, while controllers ensure that the flight system remains stable and tracks the desired setpoints accurately. The system makes use of various low-pass, notch, and dynamic filters that are designed to optimize the control process. The controllers, such as the PI and D components, are then tuned to achieve the desired balance between reactivity and stability. For further insights, see the Filters and Controllers documentation.

6.2.4 Compensation of the I-Term

The integral term in a PID controller addresses long-term errors, ensuring steady corrections over time. However, it can become problematic during quick stick movements, where it may overcorrect and destabilize the system. To avoid this, compensation methods adjust the influence of the I-term based on the situation, suppressing it during rapid changes while allowing it to integrate errors during slower movements. This approach helps maintain both precision and responsiveness. Have a look at Compensate I-Term documentation to see how we added compensation to the calculation.

7 Challenges and Methodological Approach

7.1 Simulations

At the beginning of our thesis, we developed several simulations to deepen our understanding of the existing code and the methods applied within it. Although creating these simulations was time-intensive, they proved essential for our work. They provided the foundation for all subsequent analysis steps and enabled us to fully understand the existing functions before applying them further.

7.1.1 Simulation of Betaflight Controller Order

To gain a deeper understanding of the Betaflight control architecture and its internal components, we reconstructed the complete control structure in MATLAB. The objective was to simulate the system's step response using the same PID parameters and filter settings as those used on the actual drone.

In the first step, we implemented the various filters according to the documentation Filter and Controller. For each filter type, we created continuous and discrete versions, allowing us to compare their behavior directly. An example of this comparison is illustrated in Figure 4.

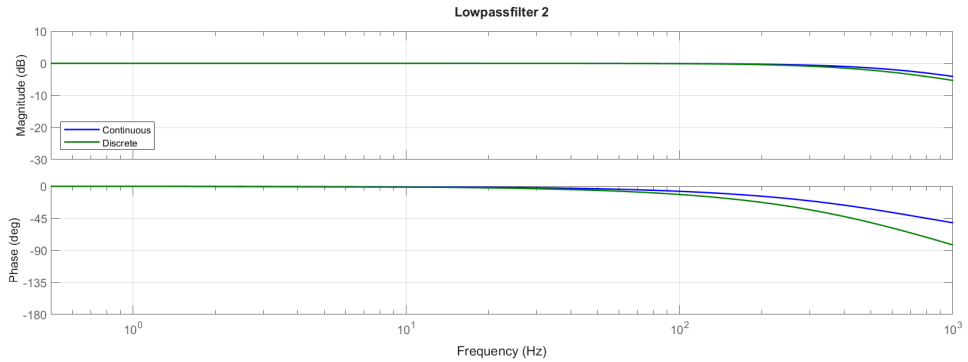

Figure 4: Comparison of continuous and discrete filters

Next, we implemented the PI and D controllers following the same principles. For the plant model, we used a simplified structure consisting of PT1 elements and a dead time, resulting in a transfer function that approximates the real system dynamics. The resulting simplified plant is shown in Figure 5.

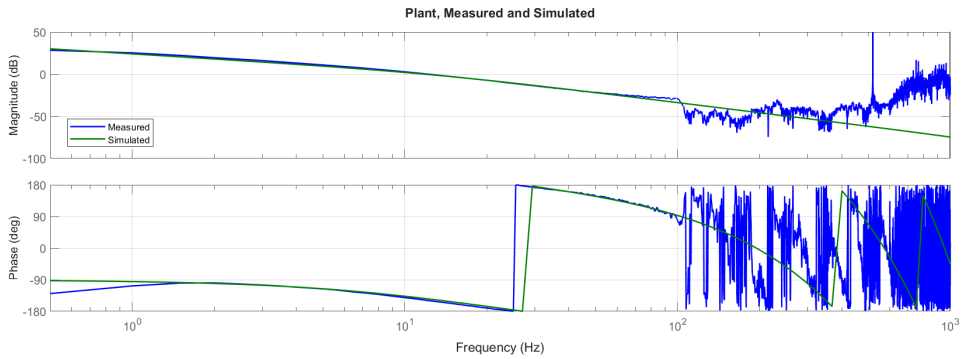

Figure 5: Simplified plant model

All components were then integrated into a complete Simulink model representing the Betaflight gyro control loop, as shown in Figure 6. Using this setup, we simulated the system's step response. An additional objective of the simulation was to identify which signals in the Blackbox log correspond to the various states within the control loop.

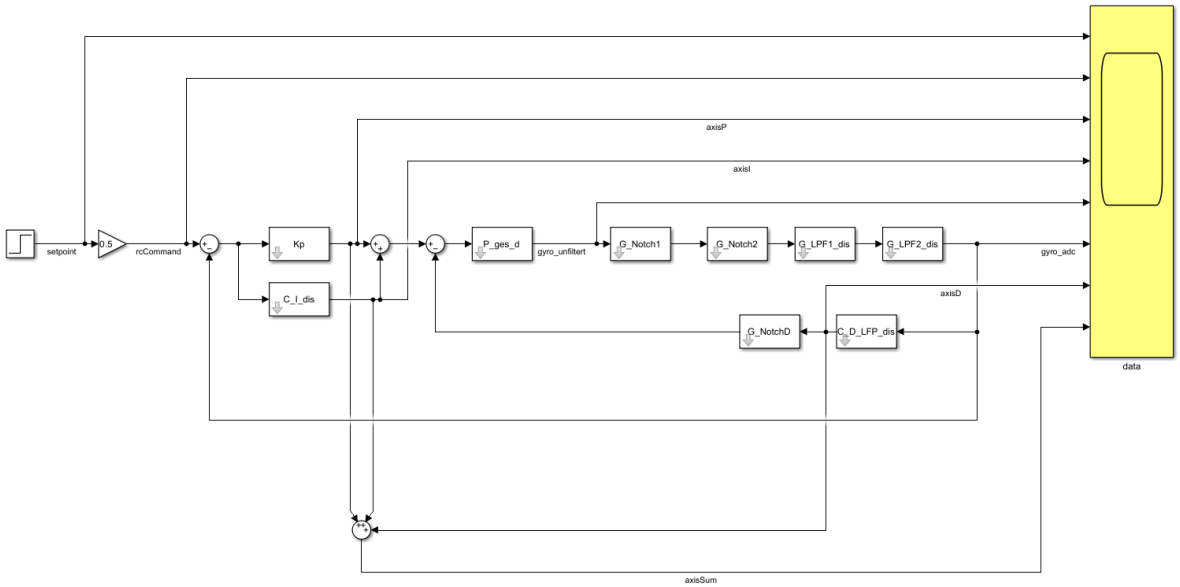

Figure 6: Simulink model of the Betaflight control system

The simulated step response, shown in Figure 7, closely matched the response of the real drone. This result confirmed that our implementation captured both the structure and the dynamic behavior of the Betaflight control system sufficiently well. Working through this simulation allowed us to check whether we had understood the control loop correctly and gave us a reliable starting point for the later calculations.

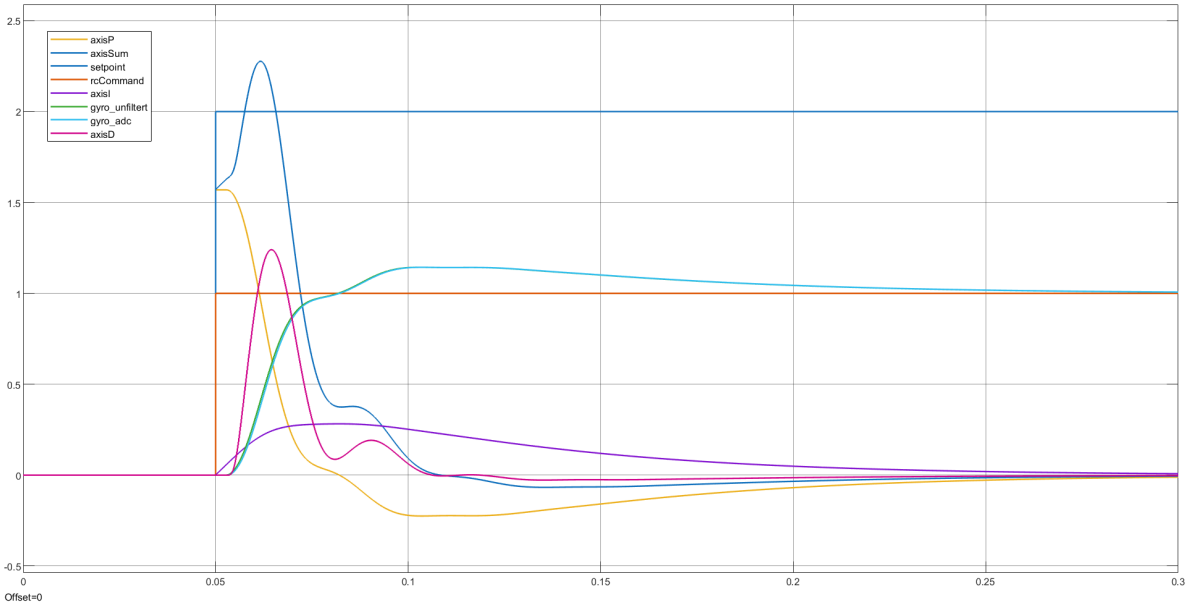

Figure 7: Simulated step response

7.1.2 Simulation PID Controller

To get a better understanding of the individual controller components, we wrote an additional MATLAB script that lets us combine different controller types with simple plant models. The user can switch between P, I, PI, PD, and PID controllers, as well as between a few basic plant types. This makes it easier to see how each controller part affects the overall system behaviour. The script calculates the open-loop and closed-loop transfer functions and produces Bode plots, Nyquist diagrams, and step responses. Figure 8 shows an example Bode plot from this simulation.

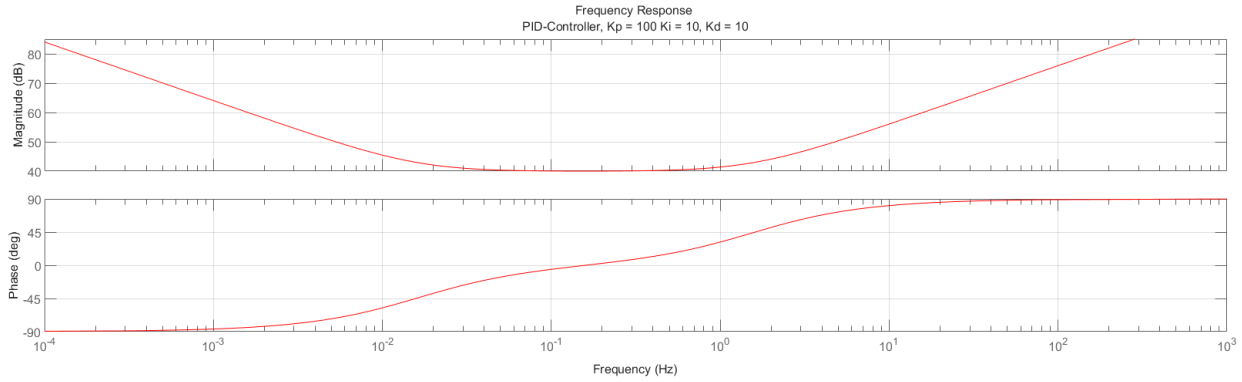

Figure 8: Bodeplot out of Controller Simulation

The frequency response of the shown PID controller is based on the parameters $K_p = 100$, $K_i = 10$ and $K_d = 10$. The proportional gain defines the magnitude level in the mid-frequency range:

$$20 \log_{10}(K_p) = 20 \log_{10}(100) = 40\,\text{dB}.$$

The integral term causes the slope of $-20\,\text{dB/decade}$ at low frequencies with

$$|C_I(j\omega)| = \frac{K_i}{\omega} = \frac{10}{\omega}.$$

The corresponding lower corner frequency is

$$\omega_{k,I} = \frac{K_i}{K_p} = \frac{10}{100} = 0.1\,\text{rad/s}.$$

The derivative term causes the slope of $+20\,\text{dB/decade}$ at high frequencies with

$$|C_D(j\omega)| = K_d\,\omega = 10\,\omega.$$

The corresponding upper corner frequency is

$$\omega_{k,D} = \frac{K_p}{K_d} = \frac{100}{10} = 10\,\text{rad/s}.$$

This example also shows that changing the integral and derivative gains shifts the position of the slopes in the Bode plot. A larger integral gain moves the low-frequency slope towards higher frequencies, while a larger derivative gain moves the high-frequency slope towards lower frequencies. In this way, the shape of the frequency response can be adjusted to the desired system behaviour.

7.1.3 Simulation Frequency Response

To analyse the frequency response of a system using input and output data, we created an additional simulation. In this setup, a chirp signal was used as the input signal. As in a real system, we added noise to both the input and the output to mimic realistic measurement conditions. From these noisy signals, we attempted to estimate the original system transfer function. The corresponding control loop is shown in Figure 9.

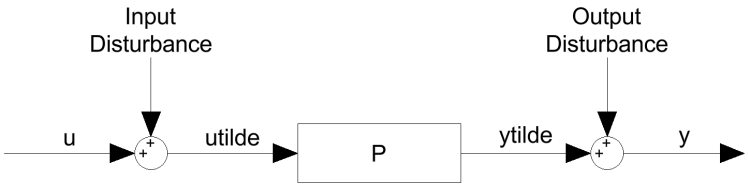

Figure 9: Control loop of the simulation

Same as in the real system, we could not measure the plant without noise. Instead, we had to measure $u(t)$ and $y(t)$ and estimate the transfer function from these signals, and not $\tilde{u}(t)$ and $\tilde{y}(t)$, which would allow us to calculate the transfer function directly. The different signals are shown in Figure 10.

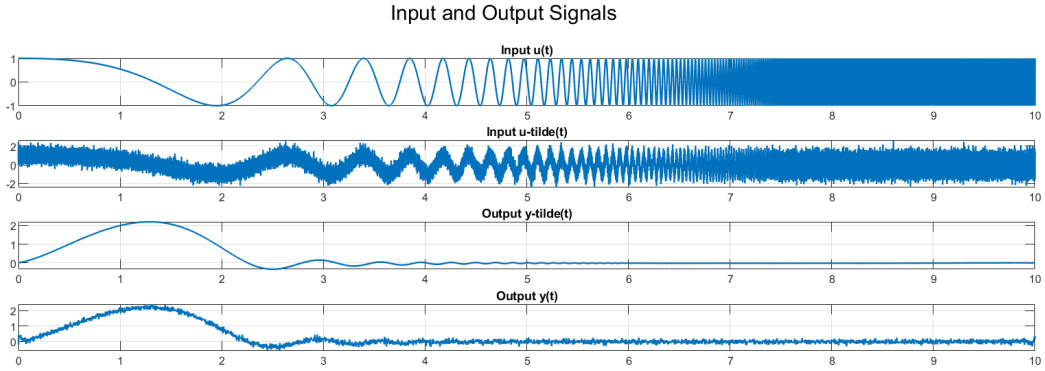

Figure 10: Input and output signals with noise

If we calculate the transfer function directly from the noisy signals using a plain FFT, we obtain a very poor estimate, as shown in Figure 11. The noise distorts the frequency response significantly.

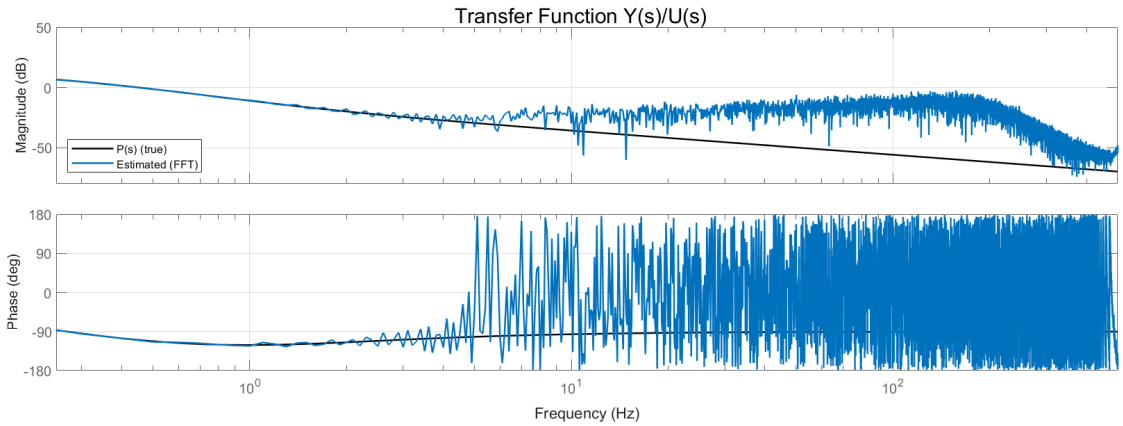

Figure 11: Bode plot estimated with plain FFT

To improve this poor estimate, we applied different signal-processing methods. First, we used the Welch method, which improves the estimation by averaging over overlapping segments. The implementation of the Welch frequency response function (FRF) follows the theory described in the documentation Estimate Frequency Response. This allowed us to compare the plain FFT approach with the more robust Welch estimator. The result is shown in Figure 12.

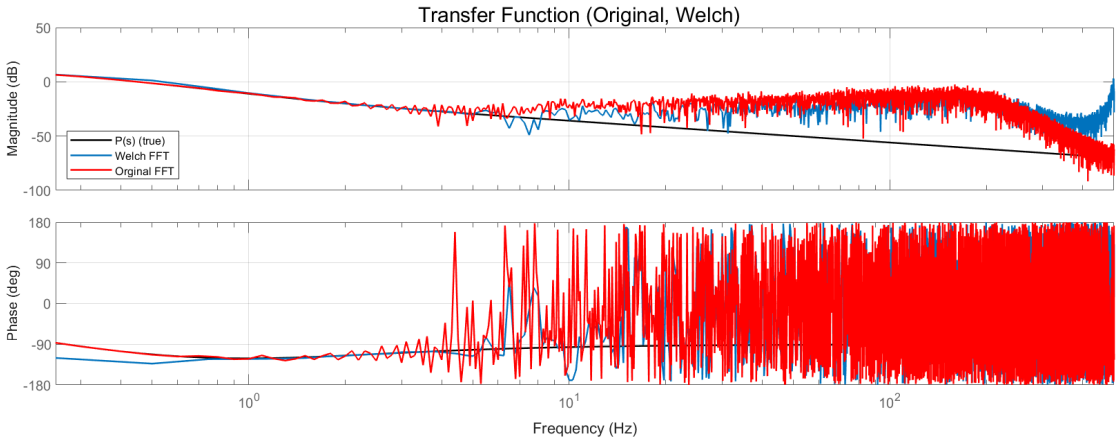

Figure 12: Bode plot estimated with Welch method

In the next step, we also implemented a rotated-signal approach, where the chirp is demodulated to baseband before filtering. For this part, we used the zero-phase filtering method `apply_rotfiltfilt`, as described in the documentation Apply Rotfiltfilt. Finally, we compared the original Bode plot to the signal-processed one, as shown in Figure 13, to assess how the rotation technique improves noise robustness.

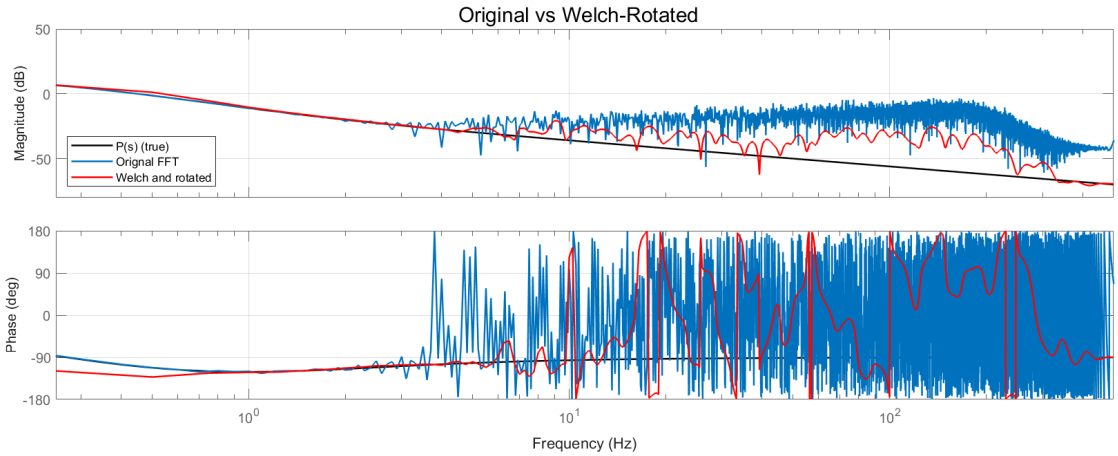

Figure 13: Bode plot estimated with rotated Welch method

7.1.4 Simulation Spectra

To better understand the components required to generate a meaningful spectrum, we developed a simulation based on the system used in the provided code. In this script, we generated a noisy sinusoidal input signal and computed different spectra from it. In one case, a window function was applied, while in the other case no windowing was used, as illustrated in Figure 14.

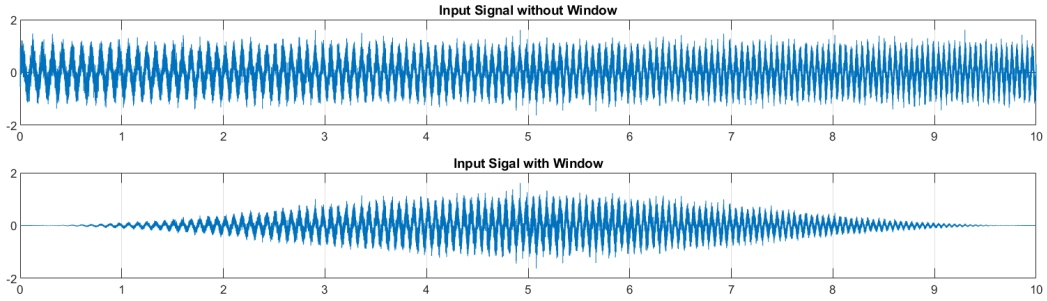

Figure 14: Spectrum with and without windowing

In the next step, we compared the frequency spectrum of the signal with and without the window. As shown in Figure 15, the spectrum without windowing exhibits strong spectral leakage, which appears as energy spreading around the main frequency peak and as several side peaks that distort the actual frequency content. In contrast, the spectrum with the Hann window applied shows much less leakage, resulting in a clearer and more accurate representation of the signal's frequency components.

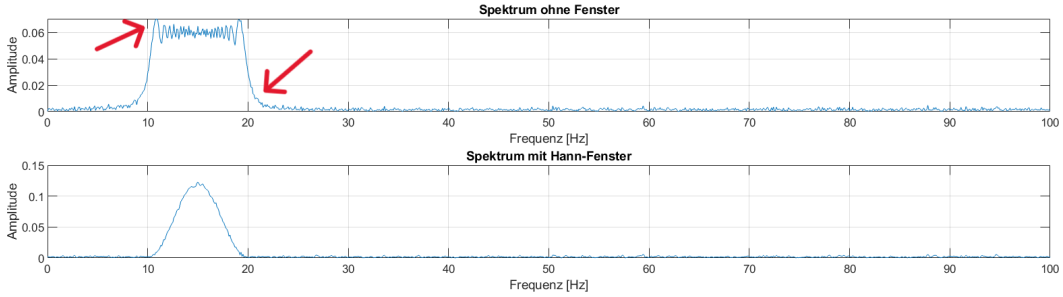

Figure 15: Comparison of Spectra with and without windowing

In the last step, we split the signal into several overlapping segments, as described in the documentation Spectra Analysis. By averaging across these segments, we were able to observe how this method reduces noise and stabilises the resulting spectrum.

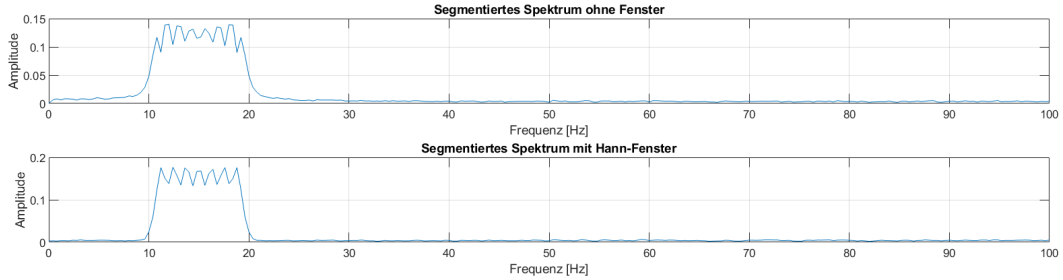

Figure 16: Spectrum with segment averaging

7.1.5 Simulation Spectrogram

To analyse how the frequency content of an input signal changes with both time and thrust, we developed a MATLAB simulation. In this script, we generated a noisy sinusoidal input signal with a slowly varying instantaneous frequency, together with a slowly varying thrust signal. The thrust is modelled as a sinusoid around a mean value of 2. To illustrate the effect of windowing, a Hann window was also applied to the full input signal, as shown in Figure 17.

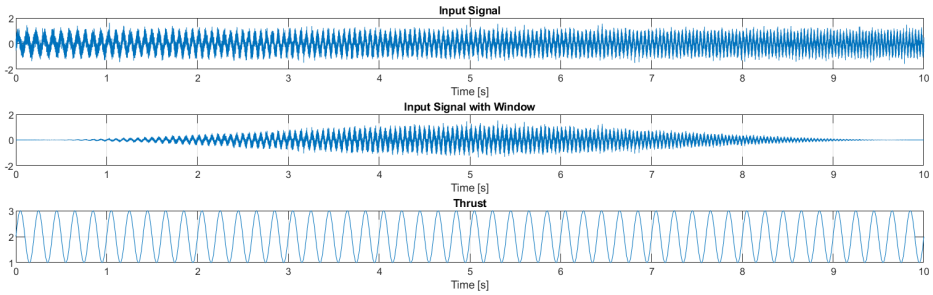

Figure 17: Input signal, windowed input signal, and thrust over time.

In the next step, the input signal was divided into short, non-overlapping time windows. For each segment, we computed the discrete Fourier transform (FFT) to obtain a local frequency spectrum. By arranging these spectra along the time axis, we created a time–frequency representation of the signal, as shown in Figure 18.

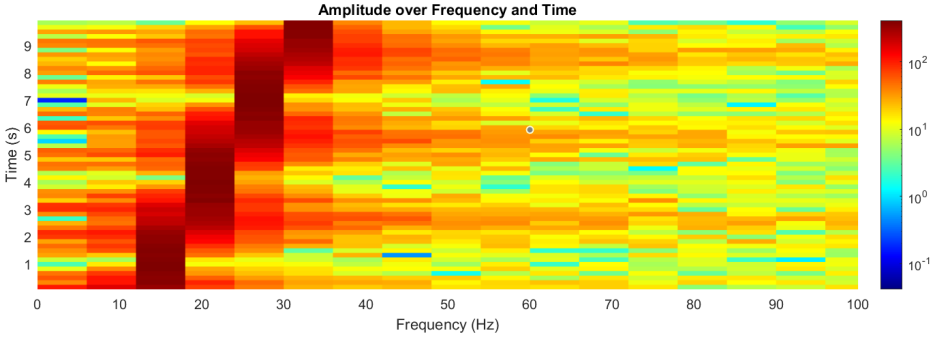

Figure 18: Spectrogram showing the amplitude over frequency and time.

To relate the spectral content to the thrust level, we assigned each segment the mean thrust value within its time interval. The corresponding spectra were then plotted over frequency and mean thrust, resulting in a spectrogram that shows how the frequency content varies with thrust for this example, as shown in Figure 19.

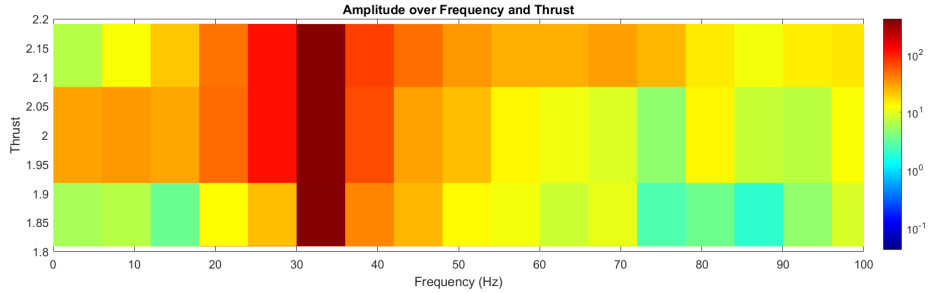

Figure 19: Spectrogram showing the amplitude over frequency and thrust.

7.2 Building an FPV drone

Besides the understanding of the code and further developing the proof of concept, building a drone and set it up correctly bridges the gap between the digital domain, where algorithms and simulations are developed and the physical reality of drone operation. By getting in touch with the hardware, it is easier for us to understand what practical challenges exist for pilots and the community.

Figure 20: aosmini 3-inch build

Therefore a three inch FPV drone was built from the ground up. This work included:

- creating schemes to see which connections were required
- soldering all the components as the motors, receiver, video system, logger and gps module to the flight controller
- attaching everything to the frame, so that nothing generates unnecessary oscillations during flight
- double check all soldering connections and screws
- setting everything up in betaflight
- testfly the drone

The flightcontroller just fitted into this frame. The GPS mount had to be trimmed off a little bit, that we could mount the all in one stack. Also the video system was mounted super close to the stack. Due to a connector between these two parts, we had to lift the videosystem up with some spacers. This resulted in not enough space on top of the videosystem for the screws, which attaches the component to the frame. The videosystem then had to be mounted with just two screws instead of four. In the first testflights, we couldn't see any too hard vibrations from this, so it's probably mounted good enough. As we need to be able to get the microSD card out of the the logger, the little pcp has to be mounted somewhere outside the drone. We designed a little 3D printed surface, where the logger could be mounted with some doublesided tape. This component then was the lowest part of the drone, which was a problem. In the first test flight we had the issue, that in

the first landing, the microSD card hold openend and we lost the card. Due to this problem, we designed some 3D printed stands for the frame, that the logger is not being crushed anymore on landings.

7.3 Tuning an FPV drone

As part of understanding the workflow we tuned an FPV drone by ourselves. The goal was to adjust the PID parameters so that we achieve a fast rising step response without overshoot, a lower compliance than before and a rounded shaped Sensitivity. Also to mention is, that the axes roll and pitch should be tuned as similar to each other as possible. This is because you want the drone to maintain a consistent and predictable response, so roll and pitch behave similarly for smoother control and a balanced flight experience.

Initially, we deliberately applied a poor tuning configuration to observe its effects. During the first field test with this setup, it quickly became apparent that the tuning was excessively poor. The drone began oscillating aggressively and climbed rapidly. As the drone was out of control, the kill switch had to be activated, resulting in a crash that broke one of the four arms. Fortunately, the damage was minor and repaired quickly.

After this incident, we decided not to repeat flights with intentionally bad tuning. Instead, we continued using the default Betaflight parameters as a baseline. With these the drone flies quite good. However, during demanding manoeuvres such as flips or propwash manoeuvres, it becomes evident that there is room for improvement in terms of stability and responsiveness.

The tuning process was carried out on a larger 6-inch FPV drone provided by one of the team members. The drone was freshly flashed with Betaflight v12, meaning all filters and settings were at factory defaults. Achieving an optimal tune required several attempts due to challenges with filter adjustments and occasional logging issues. Nevertheless, the drone was successfully tuned, and the improvement was noticeable. During propwash manoeuvres, the drone remained significantly more stable, and flips were executed with greater precision at the stop.

Our tuning objective was to achieve a fast step response without overshoot, while maintaining good compliance and sensitivity. It is important to note that there is no universally “perfect” tune. Settings depend heavily on pilot preferences. Racing pilots, for example, may want more responsiveness for competitive performance, whereas freestyle pilots want smoothness and flight feel.

Since none of our team members are professional pilots, we focused on achieving robustness and a step response without overshoot. This tuning approach is probably somewhere between what a racing pilot and a freestyle pilot would prefer. Robustness in this context is important for practical scenarios, such as when a propeller is slightly bent, the battery sits slightly off center or when an additional component like a camera is mounted on the drone.

7.4 The tuning process using our tool wird zusammengefügt mit dem oben

overview plots

Detailed information on how to interpret the generated figures can be found in the Descriptions Figures.

7.4.1 filter tuning

Before starting to tune the PID controllers, we want good filtering, that we have clean signals to work with. Therefore we started to look at the spectrograms and the spectra figures first, after the import of a log file into MATLAB. Further Information about Spectra and Spectrograms can be found here.

These figures show the noise and vibrations of the drone before and after filtering.

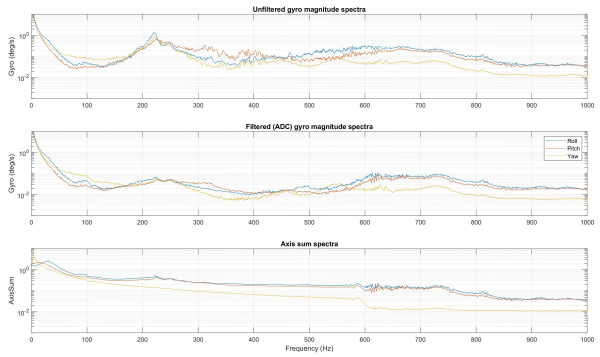

Figure 21: Spectra

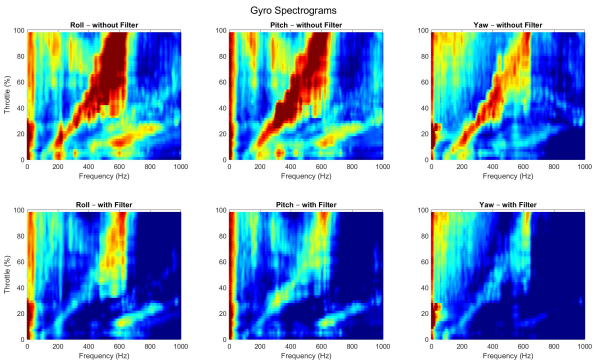

Figure 22: Spectrograms

The **spectra** in Figure 21 show the frequency content of the gyro signals. Peaks in this plot point to vibration sources at specific frequencies. When we analyze this plot, we look for three main things:

- **Resonance Peaks:** If all three axes show a peak at the same frequency, it usually means there is a resonance in the frame or a poorly fixed component. This can be fixed with a specific notch filter.
- **Noise Floor:** A high noise level suggests general vibration issues. A low noise floor means the filtering is working well.
- **Axis Sum behavior:** A well-filtered axis-sum signal should drop down quickly after the motor frequencies and stay low.

By comparing the filtered and unfiltered plots, you can see how well the current settings filter out unwanted vibrations.

The vibrations are also visible in the **spectrogram** plots in Figure 22. Here frequencies are plotted against thrust level and the color show the signal intensity. Red means high energy and blue means low energy.

Vertical lines in the spectrogram show vibrations at fixed frequencies, independent of the thrust. These are usually caused by frame resonances or loose components mounted to the frame.

Diagonal lines show vibrations that change with motor speed, such as propeller noise. Using the RPM Filter, these propeller vibrations can be filtered out quite well. This filter uses notch filters and the RPM telemetry data to remove this noise effectively. Consider looking at the Betaflight wiki for more information about the RPM filter.

The goal of all this, is to use as less filtering as possible, as too much filtering causes delay in the system. However no quadrocopter build is perfect, so at least some filtering is always necessary in order to get a good tuned system.

As we use a 2kHz logging frequency, we have set a PT1 lowpassfilter to avoid aliasing. This can be seen in the Spectrogram as a diagonal lines going down, as throttle increases The cutoff frequency should not be larger than sampling frequency divided by two. To be on the safe side we set it to 800Hz.

RPM filter
Dynamic notch filter
D term lowpass filter
Yaw lowpass filter

7.4.2 PID tuning

After we set our filter settings, we could move on to the PID tuning.

The tool calculates the Plant and Controller Bode plots to build a mathematical model of the drone, but we did not use them directly for tuning. For a pilot, these are less intuitive. Instead, we focused on the Step Response and the Gang of Four, as these directly show flight performance and stability. Detailed description about the figures can be found here.

First of all, we used the **step response** figure to determine the "feel" of the drone. The Step Response translates the complex frequency math into a simple time-domain graph. It shows exactly what happens when the pilot moves the stick rapidly.

- Tracking (Top Plot): This shows how the drone follows a command. We looked for a line that rises quickly (responsiveness) but does not overshoot. If the line rose too slowly, we increased P. If it overshot or wobbled at the top, we increased D.
- Compliance (Bottom Plot): This shows how the drone reacts to external hits, like wind. We aimed to keep this curve as low and flat as possible, which means the drone resists external forces well.

More details can be found in the Step Response documentation.

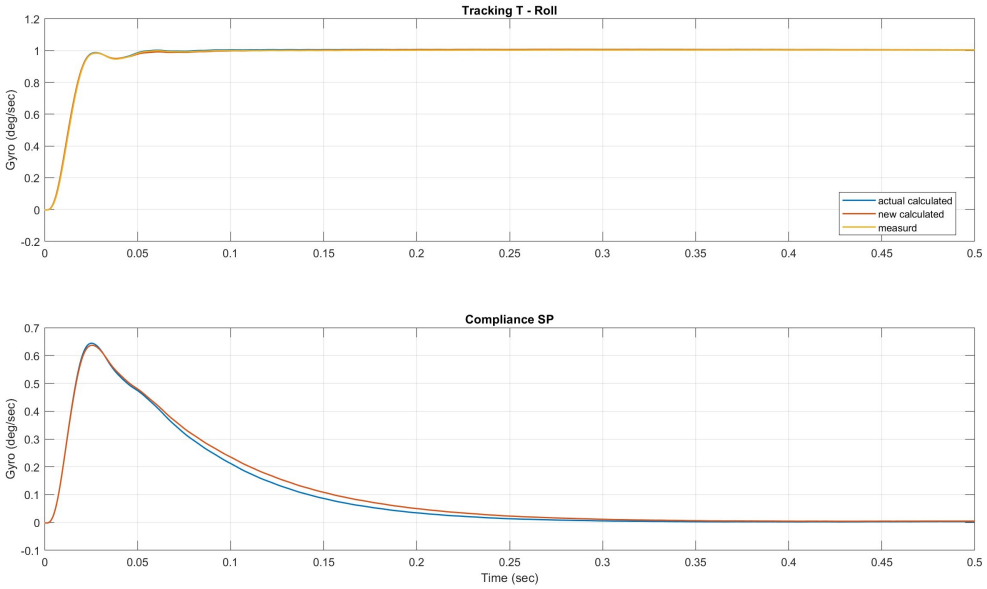

Figure 23: Step Response

7.4.3 Gang of Four

The **Gang of Four** While the Step Response showed us performance, we used the Gang of Four to ensure safety and stability. These plots break down the closed-loop behavior into four critical aspects.

During our tuning, we paid specific attention to the three sub-plots:

1. Sensitivity (S): This indicates how close the drone is to becoming unstable. We ensured that the peak in this graph stayed below 6 dB. A higher peak means the drone is on the edge of oscillating.

2. Controller Effort (SC): This shows how hard the motors have to work to follow the PID commands. We checked the high frequencies in this plot. If the curve is too high here, it means the D-term is reacting too strongly to noise, which leads to hot motors.

3. Compliance (SP): It is showing ...

By balancing these plots, we found a tune that was responsive (good Step Response) but still safe for the motors (good Controller Effort).

For a detailed explanation of the control relationships, refer to the Calculate Closed Loop documentation. Practical tuning guidance can be found in the section Gang of Four Figures.

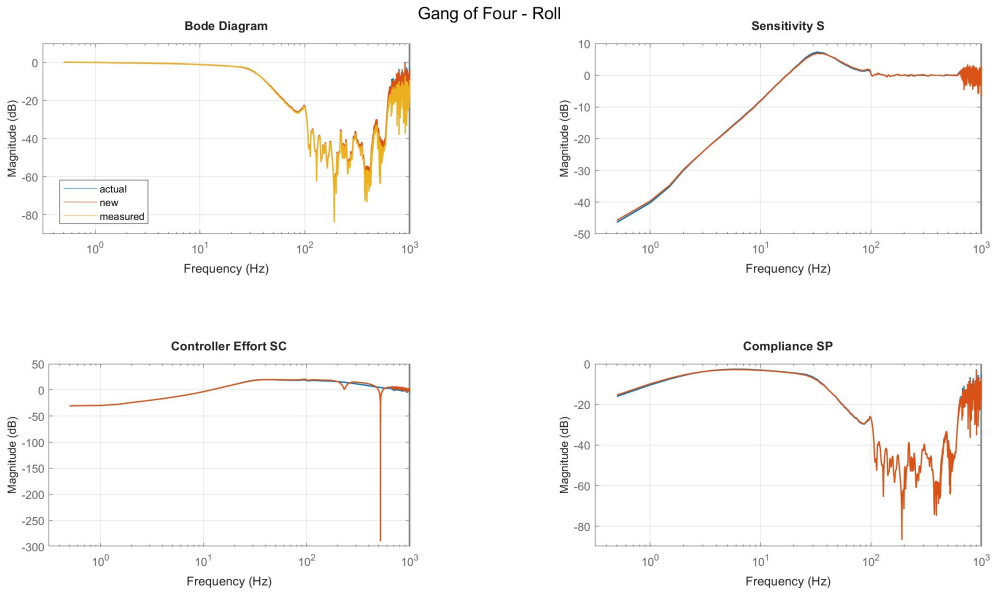

Figure 24: Gang of four

7.5 Decoding Betaflight Blackbox Logs

7.5.1 Purpose

Currently to obtain a useable Blackbox log from Betaflight, pilots rely on the Betaflight Explorer. Wich is a reliable tool, but lacks comfort if the tuning tool is used iteratively since you have to visit the Explorer every time you want to download a new log. To improve this workflow, we tried to make use of the existing open-source blackbox_decode tool.

7.5.2 blackbox_decode tool

The blackbox_decode tool is a C-based command-line utility designed to convert Betaflight Blackbox log files from their proprietary binary format into more accessible formats such as CSV. It offers similar core functionality to the Betaflight Blackbox Explorer but is intended for users who prefer command-line tools or need to automate log processing tasks.

However, integrating blackbox_decode into our project presented several challenges. The most obvious issue is the absence of header information by default. While this can be addressed by using the –save-headers flag, the format of these headers differs from that used by the Betaflight Explorer. This discrepancy is manageable, as we could adapt our data import functions to handle both formats.

A more significant problem is the presence of numerical differences in the decoded data. When comparing logs decoded with blackbox_decode to those processed with Betaflight Explorer, the resulting datasets differ structurally and semantically in several critical ways. Specifically, the blackbox_decode stores the vehicle attitude as Euler angles in degrees, whereas the Explorer provides quaternion-like components. With priority set on other tasks, we decided to set aside the integration of blackbox_decode for now.

7.6 Code Structure and programming style

From the start of the project, we recognized that a clear and maintainable code structure would be essential. The initial proof-of-concept code already contained a rough separation into topics such as data I/O and step response, but everything was packed into a single file, making it difficult to navigate. Our first step was to comment out the proof-of-concept code and reorganize it into thematic sections. This was the foundation for a more organized layout and we came to an initial directory structure:

```
bf_controller_tuning/
├── main.m
├── data_io/
│   ├── load_data.m
│   ├── extract_header_information.m
│   └── preprocess_data.m
├── flight_parameters/
│   ├── get_pid_parameters.m
│   ├── scale_pid_parameters.m
│   └── print_parameters.m
├── spectrogram/
│   ├── calculate_spectrogram.m
│   ├── plot_spectrogram.m
│   └── show_spectrogram.m
├── spectra/
│   ├── estimate_spectra.m
│   ├── plot_spectra.m
│   └── show_spectra.m
├── frequency_response/
│   ├── estimate_frequency_response.m
│   ├── plot_bode.m
│   └── show_frequency_response.m
├── controller_analysis/
│   ├── calculate_closed_loop.m
│   ├── calculate_step_response.m
│   └── show_controller_analysis.m
└── utils/
    ├── apply_rotfiltfilt.m
    ├── downsample_frd.m
    └── get_my_colors.m
```

This draft was never intended to be final. Rather, it served as an initial attempt to bring clarity and separation between different functional areas.

7.6.1 OOP vs. Functional Approach

At this stage, we discussed whether to implement the entire project using object-oriented programming (OOP) or stick to a modular, function-based approach. Initially, we considered OOP as a challenge, since most of us were not very experienced with it. We started writing some components as classes, but quickly realized that only one team member was really comfortable with OOP, while the others struggled. This made us reconsider. Our conclusion was to use OOP selectively—only for parts that benefit from reuse and encapsulation, such as data loading and plotting, while keeping the rest function-based for simplicity. This hybrid approach allowed us to maintain flexibility without overcomplicating the design.

After several attempts, we agreed on the following principles:

- A main script (main.m) for user input, figure selection, filter settings, and PID tuning
- A core class to handle all calculations (essentially the logic from the proof of concept, but without plotting)
- A plot utilities class to centralize all plotting functions for consistency

Existing library functions from the proof of concept were reused without modification. The thematic grouping included:

```
bf_controller_tuning/
├── class/
│   ├── main_class.m
│   └── plot_utils.m
├── lib/
│   ├── apply_rotfiltfilt.m
│   ├── calculate_closed_loop.m
│   ├── calculate_controllers.m
│   ├── calculate_step_response_from_frd.m
│   ├── calculate_transfer_functions.m
│   ├── downsample_frd.m
│   ├── estimate_frequency_response.m
│   ├── estimate_spectra.m
│   ├── estimate_spectrogram.m
│   ├── expand_multiple_figure_nr.m
│   ├── extract_header_information.m
│   ├── get_chirp_signals.m
│   ├── get_fcut_from_D_and_fcenter.m
│   ├── get_fcut_from_exp.m
│   ├── get_filter.m
│   ├── get_ind_eval.m
│   ├── get_my_colors.m
│   ├── get_notch_Q.m
│   ├── get_pid_scale.m
│   └── get_switch_case_text_from_para.m
├── logs/
└── main.m
```

This structure worked well and kept the main script simple for the user. However, after a review meeting, our instructor suggested further modularization. The reason: we had created a "god class," which limited flexibility. For example, if someone wanted to use the tool only for plotting spectrograms without any drone-specific logic, the current design made that difficult. Also the program was slow. All calculations were made, every time the user changed something. We wanted to improve the speed in only doing these calculations again, which are relevant on that specific user input change.

7.6.2 Final Structure Concept

Out of this, we decided to split up this god class into three classes. The `flight_analyzer` class handles frequency domain analysis and vibration characterization. Its core responsibility is performing spectral analysis on flight log data to identify resonance frequencies and assess filter effectiveness. It computes power spectral density of gyro signals (both filtered and unfiltered) and generates spectrograms showing how vibration frequencies vary with throttle levels across all three axes (roll, pitch, yaw). This analysis is essential for identifying noise sources, evaluating filtering strategies, and optimizing drone stability during tuning. The `flight_data` class is responsible for loading, parsing, and managing flight log data. It provides methods to extract relevant information from Blackbox logs, such as gyro and motor data, and prepares this data for analysis. The `gyro_ctrl_tuning` class focuses on tuning the PID controllers for the drone's flight. It uses the results from the frequency response and spectral analysis to adjust the PID parameters, aiming to optimize the drone's flight characteristics. The `plot_utils` class remains responsible for visualization and graphical representation of all analysis results. Unlike the previous approach, this class is now more flexible and reusable. It can be used independently to generate plots from pre-calculated data, making it suitable for users who only want to visualize spectrograms, frequency responses, or step responses without running the full analysis pipeline. This modularity allows the plotting functionality to be easily adapted for different visualization needs and integrated into other workflows.

This approach still holds thematic sections, but is more accessible to use only specific parts of the tool.

```
bf_controller_tuning/
├── class/
│   ├── flight_analyzer.m
│   ├── flight_data.m
│   ├── gyro_ctrl_tuning.m
│   └── plot_utils.m
└── lib/
    ├── apply_rotfiltfilt.m
    ├── calculate_closed_loop.m
    ├── calculate_controllers.m
    ├── calculate_step_response_from_frd.m
    ├── calculate_transfer_functions.m
    ├── downsample_frd.m
    ├── estimate_frequency_response.m
    ├── estimate_spectra.m
    ├── estimate_spectrogram.m
    ├── expand_multiple_figure_nr.m
    ├── extract_header_information.m
    └── get_chirp_signals.m
```

```
    ├── get_fcut_from_D_and_fcenter.m
    ├── get_fcut_from_exp.m
    ├── get_filter.m
    ├── get_ind_eval.m
    ├── get_my_colors.m
    ├── get_notch_Q.m
    ├── get_pid_scale.m
    └── get_switch_case_text_from_para.m
├── logs/
└── main.m
```

7.7 Conversion from MATLAB to Python

To make the project more accessible to the FPV community, we decided to migrate the existing MATLAB codebase to Python. The first step was to select suitable Python libraries that offer similar functionality to MATLAB, while maintaining code efficiency and readability. After evaluating several options, we chose the following libraries:

- **NumPy**: For numerical operations and array manipulations, analogous to MATLAB's matrix operations.
- **SciPy**: For scientific computing tasks, including signal processing functions comparable to MATLAB's capabilities.
- **Matplotlib**: For plotting and visualization, providing features similar to MATLAB's plotting tools.
- **Pandas**: For data manipulation and analysis, useful for handling tabular data as in MATLAB tables.
- **Control Systems Library (control)**: For control system analysis and design, offering functions equivalent to MATLAB's Control System Toolbox.

We selected Python version 3.12, as it is well-supported and compatible with the required libraries. To validate these choices, we developed a prototype script that implements the core functionalities needed for the Drone Tuning Tool. The script loads flight log data from CSV files, parses and caches the data efficiently, and extracts relevant signals such as setpoints and gyro measurements. NumPy and Pandas are used for numerical and tabular data handling, while SciPy provides signal processing utilities for windowing and frequency analysis. The Control Systems Library is employed to estimate the system's frequency response and step response from flight data, replicating MATLAB's identification routines. Matplotlib is used for visualizing raw signals, chirp responses, and system characteristics. The modular structure, including custom data classes and caching, ensures fast and reproducible analysis. This approach demonstrates that Python and its scientific libraries are suitable replacements for MATLAB in the drone tuning workflow. While Pandas is not strictly necessary for the prototype, it simplifies data handling and improves code readability. However, to minimize dependencies and ensure cross-platform compatibility, we may consider removing Pandas in the final implementation. The flexibility of Python also allows for easier integration with other tools and automation of data processing tasks, which is beneficial for large-scale log analysis.

7.8 Python in MATLAB

In our initial porting attempt, we utilized MATLAB's built-in functionality to execute Python code directly from within the MATLAB environment. While this approach allowed us to access Python libraries and scripts, it introduced some challenges. One major issue was the frequent need for unit conversions between MATLAB and Python data types, which often led to subtle bugs and inconsistencies. Additionally, we encountered differences in the implementation of certain algorithms, such as varying scaling conventions in the state-space representation of transfer functions. These discrepancies complicated the integration process and made it difficult to ensure consistent results across both platforms. Managing library dependencies and Python environments within MATLAB also proved cumbersome, especially when working across different operating systems or MATLAB versions. As a result, this method proved unreliable for our project's requirements. We ultimately decided to pursue a native Python implementation to avoid these integration issues and streamline

the development workflow.

7.9 Transferring the MATLAB Library to Python

The core functionality of the Drone Tuning Tool resides in its library functions. To ensure that the Python implementation is both reliable and consistent with the original MATLAB code, we systematically transferred each library function from MATLAB to Python. For functions with simple numerical outputs, we used `np.testing.assert_allclose` with appropriate absolute and relative tolerances, saving the function inputs and outputs from MATLAB and comparing them with the Python implementation. For more complex functions, such as those involving state-space outputs, we developed dedicated test scripts to compare the behavior of the MATLAB and Python versions across a range of scenarios. This rigorous testing approach helped us validate the correctness and consistency of the ported library functions. Automated testing and comparison between MATLAB and Python outputs were essential to ensure that the new implementation maintained the expected performance and accuracy. This systematic transfer and validation process significantly reduced the risk of introducing errors during migration and increased confidence in the reliability of the Python-based tool.

8 Results

Throughout the project, we achieved a range of practical and technical results. The most important outcome was gaining a deep understanding of the Betaflight control system, including how its filter structures and controller components interact to stabilise the gyro control loop. In addition, we developed a solid foundation in the application of essential signal-processing methods and learned how to use them correctly for frequency-domain analysis, step-response estimation and data preparation. This newly acquired knowledge will be crucial for the further development of the tool and forms an important basis for the work that will follow in the bachelor's thesis. The simulations carried out during the project helped us link theoretical concepts to the real dynamic behaviour of an FPV drone and validate our understanding of the control loops.

Alongside the theoretical work, we also built and configured an FPV drone from scr atch and performed flight tests and tuning. This hands-on experience provided valuable insight into the practical challenges of drone construction, setup, flight and tuning. It helped us better understand the real-world implications of our analysis, the needs of the FPV community, and ultimately strengthened the connection between theory and practice.

On the software side, we restructured the existing prototype into a modular codebase, created comprehensive Markdown documentation on GitHub, and began transferring key functions from MATLAB to Python to enable future extensions. We also gained experience in object-oriented programming and code organisation for larger projects. In addition, we made significant progress in migrating core functions from MATLAB to Python, such as closed-loop calculations and transfer-function estimation. Although the full transfer is still ongoing, the results achieved so far provide a strong foundation for further development and refinement during the bachelor's thesis.

A particularly time-consuming part of the project was the development of an automatic extraction method for flight data directly from Betaflight Blackbox logs. Despite testing different approaches, we were not able to complete a reliable solution within the given time frame. However, the insights gained during this process form a valuable basis for further investigations, and we plan to revisit this topic as part of the bachelor's thesis.

In summary, the project demonstrates that offline analysis and tuning of Betaflight-based FPV drones provides a robust, objective and data-driven approach to optimising flight performance. The implemented methods enable the reconstruction of control-loop behaviour from flight measurements, allowing for a detailed understanding of system dynamics and the impact of controller settings. The work carried out so far lays a solid foundation for the bachelor's thesis, during which the tool will be extended with additional tuning capabilities, fully migrated to Python, and validated through further flight tests.

9 Discussion and next steps

9.1 Discussion

Overall, the work in the last 14 weeks mainly served as a foundation for the future work at the project during our Bachlor thesis rather than a finished end product. We were able to lay the groundwork for further developments. For example, we newly structured the code, which allows us to make addtions and upgrades in the code faster and clearer. But more importantly, we were able to lern much about the behaviour of the Betaflight controll system and how we have to face challenges in it. Dario Jurietti and Janick Dort also lerned how a drone flys and how to build an FPV drone. This experience helped them to better relate the abstract analysis results and behaviour in the air.

Additionally to this, we deepened our understanding how advanced signal-processing methode and more complex controll loops working. We achived this through a series of simulations which made it us easier to connect the theorie to the measuered data of real flights. Also, we improved our skills in object oriented programming and how to structer a big code. We also were able to get our first experiene in using control-engineering in Python, even though the Python implementation is not finished yet. All of this will help us to develope new features quickly and with high quality in our Bachlor thesis.

9.2 Next Steps

This project is being continued and further developed as part of a bachelor's thesis. In this context, we plan to extend the existing codebase with new features such as offline tuning for position and altitude hold. These additions will provide deeper insight into the behaviour of the corresponding control loops and allow their optimisation.

Furthermore, a full transfer of the implementation from MATLAB to Python is planned. This will make the tool more accessible to the FPV community, as Python is open source, widely used and free of charge. Our long-term goal is to enable seamless integration of the tool into existing Betaflight utilities, so that pilots can use it for offline tuning of their drones.

Finally, we intend to validate the analysis results through additional flight tests. By comparing the outcomes of the offline analysis with real flight behaviour, we aim to ensure that the methods are accurate and reliable. This will help us further refine the tool and increase its practical value for the FPV community.

9.3 Conclusion

Throughout the project, we gained a deep understanding of the Betaflight control system, learned how its filter and controller structures interact, and applied advanced signal-processing techniques for frequency-domain analysis, step-response evaluation and flight-data preparation. Simulations allowed us to connect theoretical models with observed flight behaviour, while building and test-flying an FPV drone helped us translate analytical findings into practical insights. On the software side, we restructured the existing prototype into a modular codebase and began transferring key functions from MATLAB to Python, enabling future extensions and wider accessibility. Although the fully automated extraction of flight data from Blackbox logs could not be completed, the knowledge gained provides an important basis for further investigation. Overall, the results show that offline analysis and tuning of Betaflight FPV drones offers a robust, objective, and data-driven approach

to optimize flight performance. The implemented methods enable reconstruction of control-loop dynamics from flight measurements, allowing detailed understanding of system behaviour and the impact of different controller settings. The work carried out so far lays a solid foundation for further development and refinement of the tool in our bachelor's thesis, where we will add new tuning capabilities, complete the Python migration, and validate through additional flight tests.

References

[1] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Chirp-Signal*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Sinarg/Chirp.md.

[2] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Loading and Header Parsing for Betaflight Blackbox Logs*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Data/Dataimport.md.

[3] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Unscale high Resolution*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Data/Unscale_high_Resolution_Gain.md.

[4] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Function apply_rotfiltfilt*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Frequency_Response/ApplyRotfiltfilt.md#function-apply_rotfiltfilt.

[5] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Estimate Frequency Response*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Frequency_Response/estimate_frequency_response.md.

[6] Janick Dort, Yuri Bianchi, and Dario Jurietti. *Closed-Loop Analysis*. ZHAW - School of Engineering. URL: https://github.com/InsaneBroccoli/bf_controller_tuning/blob/PA_final/markdown/Frequency_Response/CalculateClosedLoop.md.

9.0.1 Overview of Generative AI Systems

OpenAI (2025) – ChatGPT 5.1

- Available online: https://chat.openai.com/
- Functions:
  - Generation of text suggestions
  - Grammar and language correction
  - Support in writing LaTeX and MATLAB code

DeepL (2025) – DeepL Translator (Free Version)

- Available online: https://www.deepl.com/translator
- Functions:
  - Translation of text

Microsoft (2025) – Copilot AI Assistant

- Available online: https://copilot.microsoft.com/
- Functions:
  - Generation of text and summaries
  - Integration into Microsoft Office tools such as Word and PowerPoint
  - Assistance in software development (e.g., code suggestions)

Anthropic (2025) – Claude AI

- Available online: https://claude.ai/
- Functions:
  - Text generation and editing
  - Support for research tasks (e.g., summarisation and structuring)
  - Assistance in programming-related workflows

9.1 List of Figures

10 Appendix

10.1 Project Management

10.1.1 Project Assignment

aufgabenstellung.md 2025-09-15

Projektarbeit

Allgemein

Titel

Betaflight - Offline Controller Tuning

Beschreibung

Betaflight ist ein Open-Source Projekt, das sich auf die Regelung von FPV Quadcopter spezialisiert hat, dies insbesondere im Bereich von Race-Dronen. Die Firmware wird von einer grossen Community weiterentwickelt und ist auf vielen verschiedenen Hardware-Plattformen verfügbar. Betaflight ist die am häufigsten verwendete Firmware für FPV-Drohnen und wird von vielen Hobbyisten und Profis genutzt.

Für die Flugperformace ist die Qualität der Raten-Regelung von entscheidender Bedeutung. Die Regler sind dafür verantwortlich, dass die Drohne stabil fliegt, Störungen wie Wind und Strömungsabrisse (Prop-Wash) unterdrückt und möglichst agil auf Steuerbefehle reagiert. Die Regler und Filter müssen daher sorgfältig getunt und abgestimmt werden, um eine optimale Flugperformance zu erzielen. FPV Piloten fliegen üblicherweise im ACRO mode, sprich das System ist Ratengeregelt und die Stickinputs des Piloten werden als Sollwerte auf den Ratenregler gegeben. Das Tuning dieser Regler geschieht in der Community im Allgemeinen durch Trial and Error. Das bedeutet, dass die Piloten die Reglerparameter anpassen und dann testen. Dies kann jedoch zeitaufwendig und frustrierend sein, da es oft schwierig ist, insbesondere für Leien, die richtigen Parameter zu finden.

Bereits bestehend ist ein Proof of Concept, welches es ermöglicht die Raten-Regler offline basierend auf einem Messdaten-Modell der Drone zu tunen (basierend auf Chirp-Signal Experiment während dem Flug). Das Projekt soll auf diesem Proof of Concept aufbauen und die Methode des Offline Tunings für die Raten-Regler (Gyro) weiterentwickeln. Ziel ist es, eine vollständige, einfach erweiterbare und sauber implementierte Open-Source Libary zu entwickeln. Dies sowohl in MATLAB als auch in Python. Ausserdem soll für die Python Implementation eine ausführliche Dokumentation mit Beispielen für Anwender erstellt werden (Github Markdown Dokumentation). Das Ziel ist es, dass die Libary von der Community genutzt werden kann. Dies soll dazu beitragen, die Flugperformance der Drohnen zu verbessern und den Piloten eine einfachere Möglichkeit zu bieten, ihre Drohnen zu tunen.

Studenten

Bianchi Yuri (biancyur)
Dort Janick (dortjan1)
Jurietti Dario (juriedar)

Betreuer

Hauptbetreuer: Michael Peter (pmic)
Nebenbetreuer: Prof. Dr. Ruprecht Altenburger (altb)

Weitere Personen

Camille Huber (hurc)

Voraussetzungen

- GRT, RT1
- Zumindest eine Person ist im Besitz des Fernpiloten-Zeugnis A1/A3 (A2 optional jedoch empfohlen)
- Genügend hohe Haftpflichtversicherung, muss von den Studierenden selbst organisiert werden
- Erfahrung mit FPV Dronen (Bauen, Reparieren, Konfigurieren, Fliegen)
- MATLAB, Python

Vereinbarung

Die Vereinbarung wird zwischen dem Betreuer Michael Peter (pmic) und den Studierenden Yuri Bianchi (biancyur), Janick Dort (dortjan1) und Dario Jurietti (juriedar).

Da es sich um eine dreier PA handelt wird das Projekt in drei separat bewertbare Teile aufteilt:

- MATLAB Implementation
- Python Implementation
- Markdown Dokumentation mit Beispielen der Matlab und Python Implementation

Zeitplan

Siehe: https://intra.zhaw.ch/departemente/school-of-engineering/bachelorstudium/projekt-und-bachelorarbeiten

Projektarbeit HS 25	
Ab 15.09.2025	Ausgabe der Aufgabenstellung durch Dozierende an Studierende
Bis 19.12.2025 bis 18:00 Uhr	Abgabe der Projektarbeit in Complesis
03.02.2026	Prov. Notenfreischaltung für Studierende ab 18:00 Uhr
27.02.2026	Def. Notenfreischaltung für Studierende ab 18:00 Uhr

Arbeitsplatz

TE 617

Hardware

- Hardware kann von Michael Peter und IMS bezogen werden
  - 2 x 3.5 Zoll Dronen
  - 2 x 5.0 Zoll Dronen
- Es können auch noch zwei weitere Dronen (1 x 3.5 Zoll, 1 x 5.0 Zoll) gebaut werden

Ergebnisdarstellung

Ein Bericht im Umfang von 20-30 Seiten dokumentiert den Entwicklungsweg geeignet. Die Software wird in einem öffentlichen GitHub Repository abgelegt. Des weiteren soll eine Dokumentation mit Beispielen in Markdown erstellt werden. Die technische Dokumentation ist ebenso in Markdown zu verfassen.

Fachgebiet

Primäres Fachgebiet

Regelungstechnik und Advanced Control

Weitere Fachgebiete

- Datenvisualisierung
- Digitale Signalverarbeitung
- Software Engineering

Verschiedenes

Studiengang

Systemtechnik

Industriepartner

Keiner

Institut / Zentren

Institut für Mechatronische Systeme (IMS)

Interne Partner

Keine

Informationslink

https://github.com/pichim/bf_controller_tuning

Bemerkungen

- Grundsätzlich ist die Idee die PA aufbauend in einer BA zu erweitern

Sprache

Deutsch / Englisch

Studenten Detail

- Yuri Bianchi (biancyur) - Automatiker - Leader, Pilot, Regelungstechnik & Signalverarbeitung
- Dario Jurietti (juriedar) - Biolaborant - Datenverarbeitung, Software
- Janick Dort (dortjan1) - Elektroplaner - Datenverarbeitung, Pilot, Regelungstechnik & Signalverarbeitung

Die Studenten haben schon PM 1-4 zusammen absolviert und sind ein eingespieltes Team.

10.1.2 Project Schedule – Version 0

This project schedule was created at the beginning of the project.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	3														
Simulation Regler (Simulink/MATLAB)	1	3														
Codestruktur definieren	2	4														
Drohne bauen	3	5														
Milestone Meeting 1	5	5														
MATLAB nach Codestruktur (Klassen) umschreiben	4	7														
Weiterentwicklung MATLAB Code	7	9														
.txt Logfile in .csv umwandeln	9	11														
Milestone Meeting 2	9	9														
MATLAB -> Python	9	12														
Optimierung Chirp Signal Betaflight	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

Figure 26: Project Schedule – Version 0

10.1.3 Project Schedule – Version 1

This project schedule was created prior to the first milestone meeting and approved during the meeting.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	4														
Simulation Regler (Simulink/MATLAB)	1	3														
Simulation Überttragungsfunktion	2	4														
Codestruktur definieren	2	4														
Simulation PID Regler	3	4														
Simulation Spektrum	4	5														
Drohne bauen	5	7														
Milestone Meeting 1	5	5														
Simulation Spektrogram	5	6														
MATLAB nach Codestruktur (Klassen) umschreiben	5	8														
~~Weiterentwicklung MATLAB Code~~	~~7~~	~~9~~														
.txt Logfile in .csv umwandeln	1	6														
Milestone Meeting 2	9	9														
MATLAB -> Python	8	10														
Weiterentwicklung Python Code	10	13														
Optimierung Chirp Signal Betaflight	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

Figure 27: Project Schedule – Version 1

10.1.4 Project Schedule – Version 2

This project schedule was created prior to the second milestone meeting and approved during the meeting.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	4														
Simulation Regler (Simulink/MATLAB)	1	3														
Simulation Überttragungsfunktion	2	4														
Codestruktur definieren	2	4														
Simulation PID Regler	3	4														
Simulation Spektrum	4	5														
Drohne bauen	5	12														
Milestone Meeting 1	5	5														
Simulation Spektrogram	5	6														
MATLAB nach Codestruktur (Klassen) umschreiben	5	8														
~~Weiterentwicklung MATLAB Code~~	~~7~~	~~9~~														
.txt Logfile in .csv umwandeln	1	6														
Milestone Meeting 2	9	9														
Erstellung eines Miniskript in Python	7	10														
GitHub Dokumentation	8	11														
Optimierung Codestruktur MATLAB	9	11														
Umschreiben der MATLAB Libary in Python mit Funktionstest	10	13														
~~MATLAB -> Python~~	8	10														
~~Weiterentwicklung Python Code~~	10	13														
~~Optimierung Chirp Signal Betaflight~~	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

Figure 28: Project Schedule – Version 2

10.1.5 Meeting Minutes

10.2 Additional Information

Projektcode